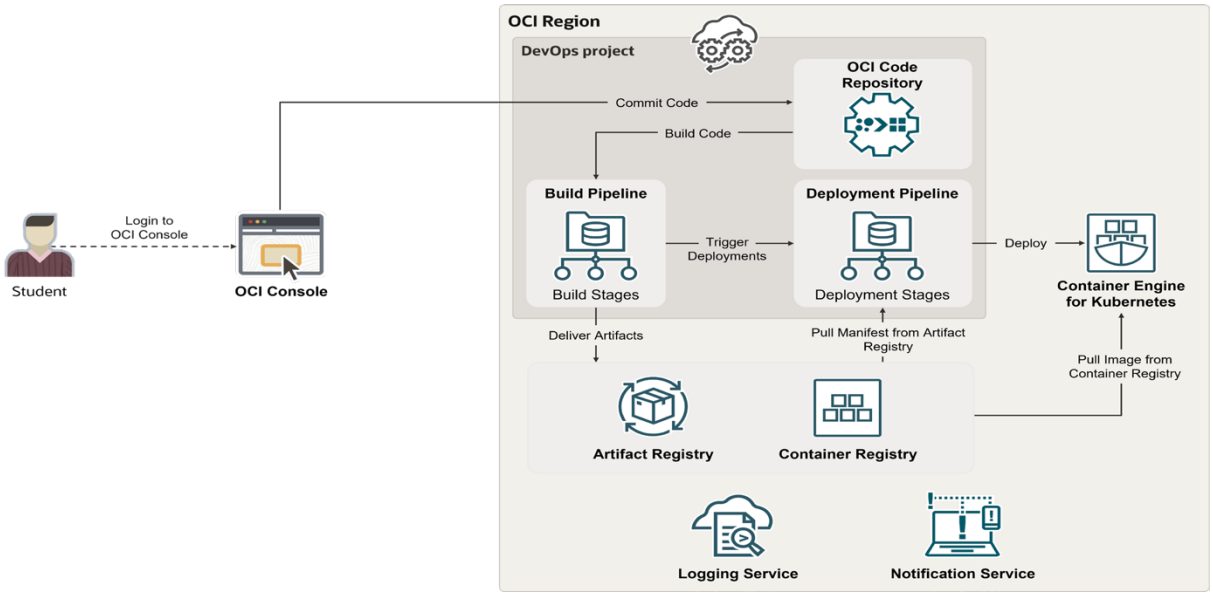
Lab 11 Build Microservices using DevOps

Overview

The Oracle Cloud Infrastructure (OCI) DevOps service is an end-to-end, continuous integration and continuous delivery (CI/CD) platform for developers.



In this lab, you will:

- Create a repository to store and manage artifacts.
- Add a Container Image Repository artifact to store your Docker images.
- Create a reference to Kubernetes manifest.
- Create a DevOps environment.
- Prepare the Kubernetes Deployment Manifest for automated deployment.
- Create DevOps build pipeline and build stages.

Prerequisites

- You are signed into your Oracle Cloud Infrastructure (OCI) account using your credentials.
- You must complete the following labs before you perform tasks for this practice:
  - *Create Docker image for a web application using Dockerfile*.
  - *Create and work with OCIR repository*.
  - *Deploy a load-balanced Web application on an OKE cluster using Kubectl*.
  - *Work with code repositories in OCI DevOps project*.
  - *Create and set up artifacts and environments in DevOps project*.

Assumptions

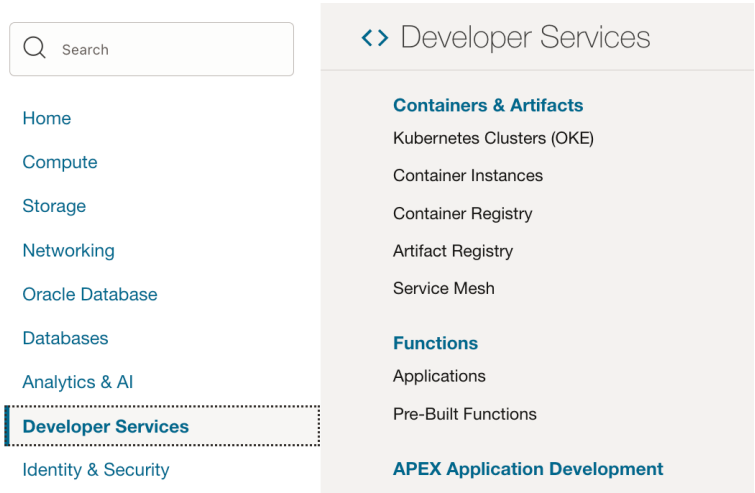
- An OCIR repo `<region-key>-lab04-ocir-1/webapp_<userID>`.

- You have an auth token `AIOPS-LAB04-AT-01`.
- You have a namespace `ns-<userID>` and Kubernetes Secret `ocir-secret-<userID>` created
- Replace the `<tenancy-namespace>` and `<username>` values from the info given in the Profile menu.

Sequence 1. Create a Repository to Store and Manage Artifacts

In this lab, you will create an Artifact Registry Repository to store your Kubernetes manifest.

1. In the Console, open the navigation menu and click **Developer Services**. Under Containers & Artifacts, click Artifact Registry.



2. Select your `<Compartment Name>` from **Applied Filters**
3. Click **Create repository** and fill the following values in the form:
   a. **Name**: `IAD-DOP-LAB11-1-AR-01`
   b. **Compartment**: Select your `<Compartment Name>`.
   c. Select **Immutable artifacts**. Your new repository will make its artifacts immutable.
   d. Click **Create.**

Sequence 2. Add Container Image Repository to Store Docker Images

Artifacts are used to specify software package versions for deployment. DevOps artifacts can be of following types:

- Container image repository
- Instance group deployment configuration
- Kubernetes manifest
- General artifact
- Helm Chart

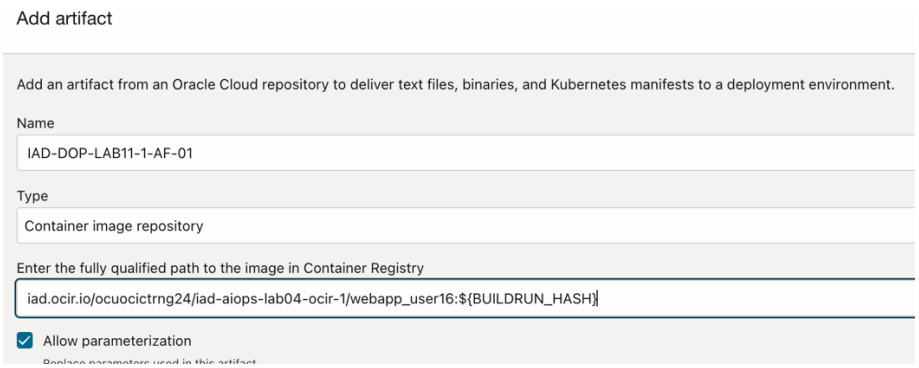
1. From the navigation menu and click **Developer Services**. Under **DevOps**, click **Projects**.
2. Select your `<Compartment Name>` from List scope on the left menu.

3. Open the DevOps project **IAD-DOP-LAB05-1-DP-01-<*userID*>** created earlier.
4. Click **Artifacts** from the left menu to navigate to the artifacts page.
5. Click **Add artifact** to create an artifact and fill the form with the following values:
   a. **Name**: `IAD-DOP-LAB11-1-AF-01`
   b. **Type**: Select **Container image repository** from the list of options.
   c. **Fully qualified path to the image in Container Registry:**

      `<region-key>.ocir.io/<tenancy-namespace>/<repo-name>:<tag>`

      For example,

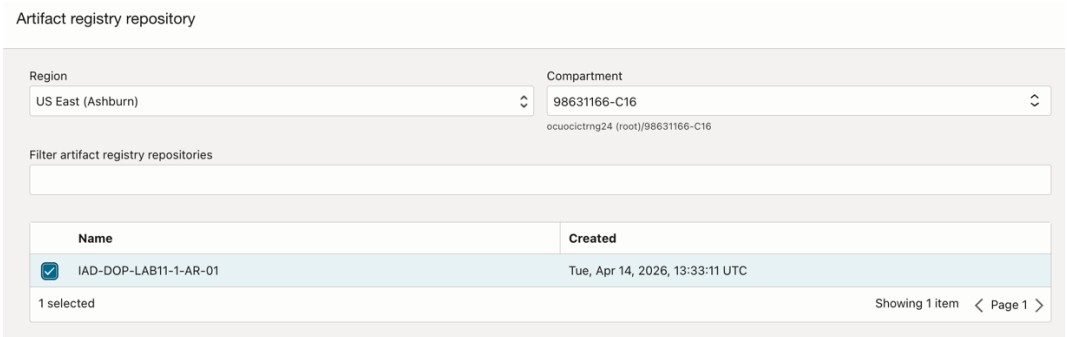
      `iad.ocir.io/ocuocictrng24/iad-aiops-lab04-ocir-1/webapp_user16:${BUILDRUN_HASH}`



      - Replace `<tenancy-namespace>` with your tenancy name and `<userID>` with your User ID. Refer to Lab 3, Sequence 6, Step 11 for exact name.
      - Ensure that you append `${BUILDRUN_HASH}` in the fully qualified image URL as the tag. This dynamically updates the version of the pushed docker image.

   d. Select **Allow parameterization** and click **Add**.
6. Click **Add artifact** to create an artifact and fill the form with the following values:

   a. **Name**: `IAD-DOP-LAB11-1-AF-02`
   b. **Type**: Select **Kubernetes Manifest** from the list of options.
   c. **Artifact Source**: Select **Artifact Registry repository**.
   d. **Artifact Registry repository**:
   e. Click **Select** and select your artifact registry **IAD-DOP-LAB11-1-AR-01** created earlier.

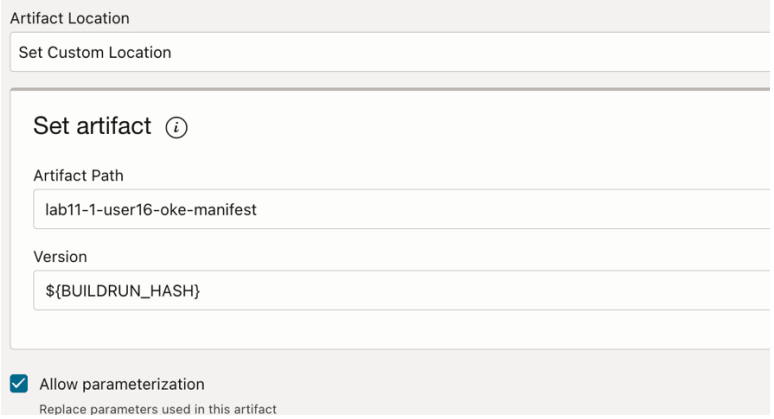


f. **Artifact Location**: Select **Set Custom Location**.
   1) **Artifact Path**: `lab11-1-<userID>-oke-manifest`

      For example,

      `lab11-1-user16-oke-manifest`

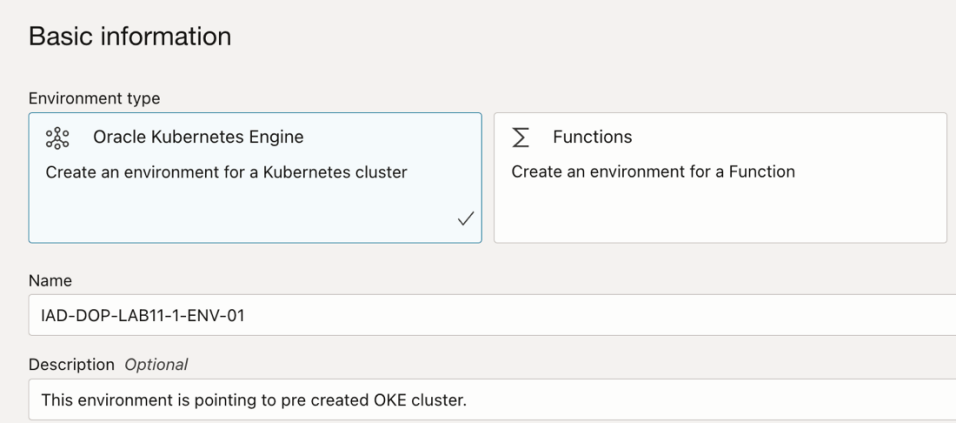
   2) **Version**: `${BUILDRUN_HASH}`
g. Select **Allow parameterization** and click **Add**.



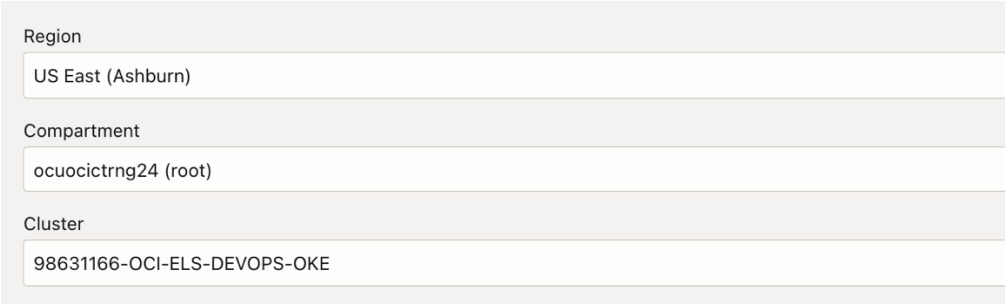
Sequence 4. Create a DevOps Environment

An environment is the target platform for your application. You will now create an Environment to point to your OKE cluster.

1. Open the DevOps project **IAD-DOP-LAB05-1-DP-01-<*userID*>**.
2. Click **Environments** from the left menu to navigate to the environments page.
3. Click **Create environment** and select **Oracle Kubernetes Engine** as the Environment Type. Fill the rest of the form with the following values:
   a. **Name**: `IAD-DOP-LAB11-1-ENV-01`
   b. **Description**: `This environment is pointing to pre created OKE cluster.`



c. Click **Next**.
d. **Region**: The region you are working in. This is populated by default.
e. **Compartment**: Select root compartment.
f. **Cluster**: Select **<EventID>-OCI-ELS-DEVOPS-OKE** from the list.



g. Click **Create environment.**

Sequence 5. Prepare the Kubernetes Deployment Manifest for Automated Deployment

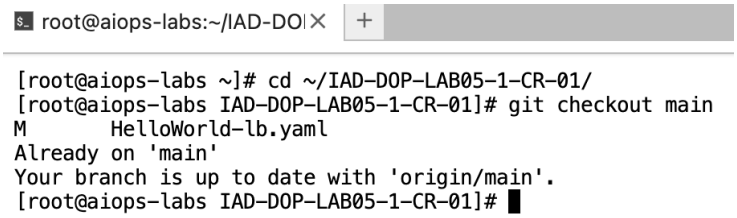
A manifest specifies the desired state of an object that Kubernetes will maintain when you apply the manifest. In this lab, you will edit the Kubernetes Deployment Manifest `HelloWorld-lb.yaml` to prepare for an automated deployment.

1. Open the terminal of your jupyter lab, switch to root user and go to the cloned OCI DevOps Code Repository.

```
sudo su -
cd ~/IAD-DOP-LAB05-1-CR-01/
```

2. Make sure that you're in the git main branch.

```
git checkout main
```



a. Edit the `HelloWorld-lb.yaml` and replace the tag `latest` with `${BUILDRUN_HASH}` in image name, as follows:

```
image: <region-key>.ocir.io/<tenancy-namespace>/<repo-name>:${BUILDRUN_HASH}
```

- `${BUILDRUN_HASH}`: This dynamically updates the tag of a Docker image pushed into OCIR to be used as a deployment artifact.

```yaml
  replicas: 1
  template:
    metadata:
      labels:
        app: helloworld
    spec:
      containers:
      - name: helloworld
    # enter the path to your image, be sure to include the correct region prefix
        image: iad.ocir.io/ocuocictrng24/iad-aiops-lab04-ocir-1/webapp_user16:${BUILDRUN_HASH}
        ports:
        - containerPort: 80
      imagePullSecrets:
    # enter the name of the secret you created
      - name: ocir-secret-user16
---
apiVersion: v1
kind: Service
metadata:
  name: helloworld-service-user16
```

3. Use `<esc> : wq` to save a quit the vi editor.
4. Commit and push your changes to OCIR IAD-DOP-LAB05-1-CR-01.
   a. Add the changes in the working directory to the staging area:

      ```
      git add .
      ```

   b. Save the changes in the staging area with a relevant message:

      ```
      git commit -m "HelloWorld-lb.yaml modified"
      ```

   c. Push the changes to the upstream code repository:

      ```
      git push -u -f origin main
      ```

   When prompted, enter your username `<tenancy-namespace>/<username>`. For example, `ocuocictrng24/98631166-lab.user16`

   Enter the auth token for the password

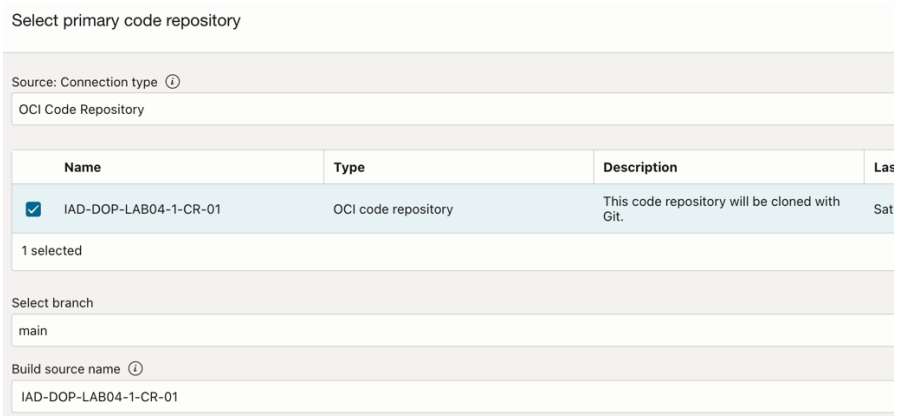
Sequence 6. Create DevOps Build Pipeline and Build Stages

A build pipeline contains the stages that define the build process for successfully compiling, testing, and running software applications before deployment. A stage is an action in the build pipeline. The OCI DevOps service includes the following predefined stages that you can use in a build pipeline:

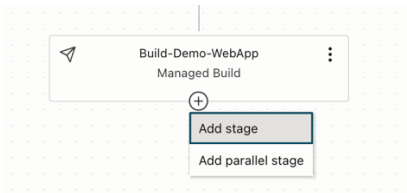
- **Managed Build**: Build and test your software applications.
- **Deliver Artifacts**: Store your software applications created from the Managed Build stage in the OCI Artifact Registry or OCI Container Registry repositories.
- **Trigger Deployment**: Start a deployment pipeline to deploy the output from the build pipeline.
- **Wait**: Pause a specific duration for testing the build pipeline.

You can add multiple stages to a pipeline. Stages can be added in a sequence or in parallel. You can remove any stage from the pipeline. When you do, the stage and its associated resources are deleted.

1. Open the navigation menu and click **Developer Services**. Under **DevOps**, click **Projects**.
2. Select your `<assigned compartment>` from the List scope on the left menu.
3. Open the DevOps project **IAD-DOP-LAB05-1-DP-01-<*userID*>**.
4. Click **Build Pipelines** from the left menu to navigate to the Build Pipelines page.
5. Click **Create build pipeline** and fill the form with the following values:
   a. **Name**: `IAD-DOP-LAB11-1-BPL-01`
   b. **Description**: `This is the Build pipeline for Lab11.`
   c. Click **Create.** The **Build pipeline** tab will open.
6. To add the first stage to the build pipeline, click the **+** icon and click **Add stage.**
   a. Select **Managed Build** as stage type and click **Next.** Fill the form with the following
      1) **Stage name**: `Build-Demo-WebApp`
      2) **Description**: `This stage executes the commands specified in build_spec.yaml file.`
      3) Default compute shape information is displayed for the OCI build agent.
      4) **Base container image**: Default is Oracle Linux 8 x86_64 standard 1:0
      5) **Connect to your tenancy subnet**: Leave this blank.
      6) **Build spec file path**: The file `build_spec.yaml` is in the root directory, so leave this field blank.
      7) **Primary code repository**: Click **Select**. This opens the window to select Primary code repository:
         a) Select the **OCI Code Repository** from the drop-down list for **Source: Connection type**.
         b) Select **IAD-DOP-LAB05-1-CR-01**. The branch and Build Source name will auto-populate.



         c) Click **Select**.
      8) **Additional code repositories**: Leave this field blank.
      9) **Timeout** (in seconds): Continue with the default value of 36000.
      10) Click **Add**. You will notice a stage with name **Build-Demo-WebApp (Managed Build)** has been added.
7. Add the second stage to the Build pipeline by clicking the **+** icon at the **bottom** of the Build-Demo-WebApp (Managed build) box and click **Add stage**.



a. Select **Deliver Artifacts** as stage type from the optional section and click **Next**. Fill the form with the following values:
   1) **Stage name**: `Push WebApp Artifacts`
   2) **Description**: `This stage uploads artifacts to registries.`
   3) Click the **Select Artifact(s)** button. Select the following artifacts and click **Add**:

      - **IAD-DOP-LAB11-1-AF-01** *Docker Image*
      - **IAD-DOP-LAB11-1-AF-02** *Kubernetes Manifest*

b. **Associate artifacts with build result**: You will provide the output names used in the **outputArtifacts** section of the `build_spec.yaml` file corresponding to the artifact types in the build config/result artifact name field.

   A snippet of the `build_spec.yaml` file:

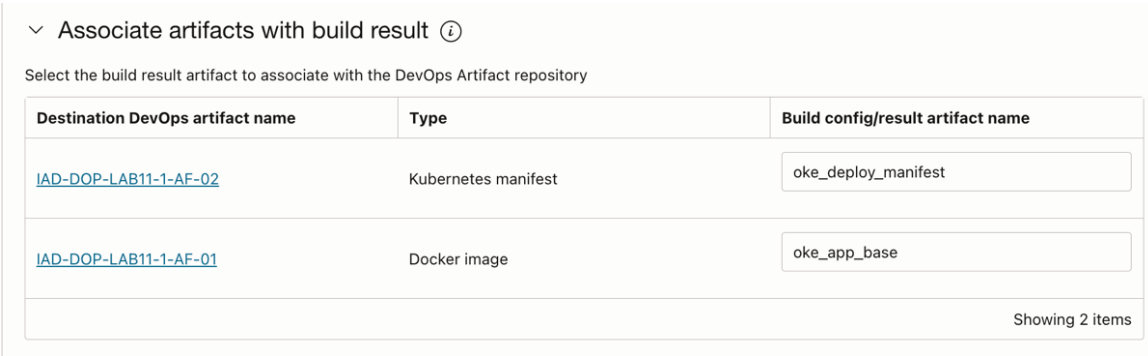
```
outputArtifacts:
  - name: oke_app_base
    type: DOCKER_IMAGE
    # this location tag doesn't effect the tag used to deliver the container image
    # to the Container Registry
    location: oke_app_base:latest

  - name: oke_deploy_manifest
    type: BINARY
    # this location tag doesn't effect the tag used to deliver the container image
    # to the Container Registry
    location: ${OCI_PRIMARY_SOURCE_DIR}/HelloWorld-lb.yaml
```

   The `build_spec.yaml` is available in the root directory of your DevOps code repository **IAD-DOP-LAB05-1-CR-01**.

   Fill the fields appropriately as shown in the following table:

Destination DevOps artifact name	Type	Build config/result artifact name
`IAD-DOP-LAB11-1-AF-01`	Docker image	oke_app_base
`IAD-DOP-LAB11-1-AF-02`	Kubernetes manifest	oke_deploy_manifest

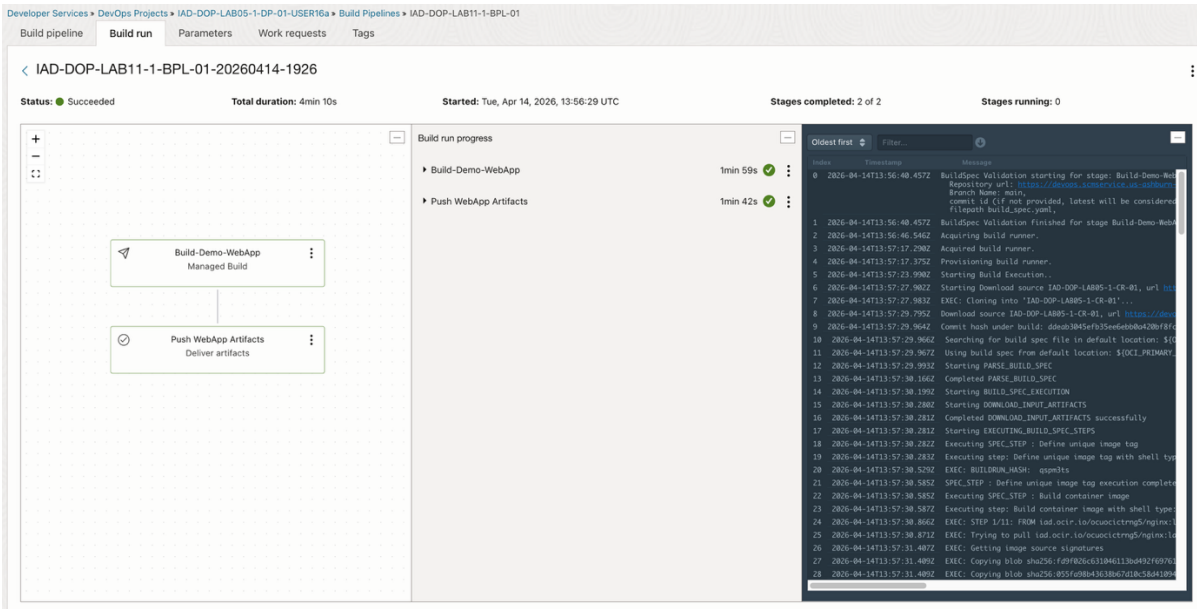


c. Click **Add**. You will notice a stage with name **Push WebApp Artifacts (DeliverArtifacts)** added. At this point you have two stages in your Build pipeline IAD-DOP-LAB11-1-BPL-01.

8. Click the **Start Manual Run** button from top right of the Build Pipeline Page. The Start Manual Run page opens.

   a. OCI assigns your build a Build run name.
   b. Click **Start manual run** at the bottom left.

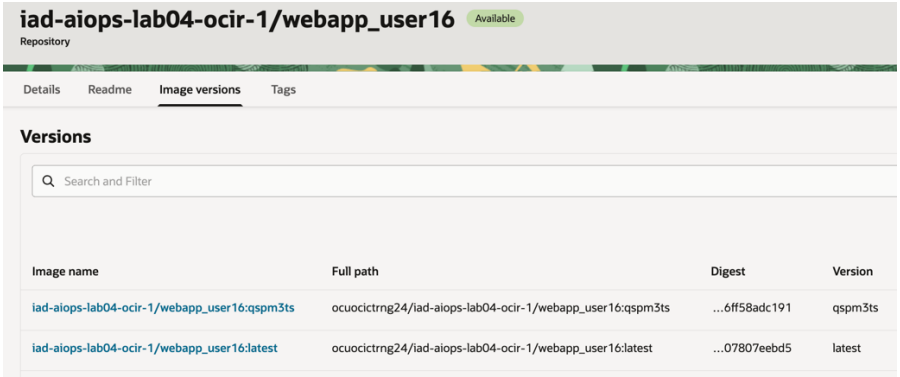
9. You will reach the **Build pipeline** tab. Observe that all the build stages are listed here. Build stages will execute sequentially. You can observe the logs for each stage in the right window.



Sequence 7. View the Artifacts Generated as Part of the Automated Build

1. To view Container Image Repository Artifact:
   a. In the Console, open the navigation menu and click **Developer Services**. Under **Containers & Artifacts**, click **Container Registry**.
   b. Select your `<assigned compartment>` from **Applied filters.**
   c. Select the repository `iad-aiops-lab04-ocir-1/webapp_<userID>` created Earlier.
   d. You will notice a new image version under **image versions** present in your repository with a

random string like `qm3pznq` as tag. This random string is the `BUILDRUN_HASH` of the build that pushed the image in OCIR.



2. To view Kubernetes manifest reference:
   a. In the OCI Console, open the navigation menu and click **Developer Services**. Under Containers & Artifacts, click Artifact Registry.
   b. Select your *<assigned compartment>* from **Applied Fileters**.
   c. Select the artifact registry **IAD-DOP-LAB11-1-AR-01**.
   d. You will notice the Kubernetes manifest used by the Deployment pipeline listed here with the same string `qm3pznq` as tag.

Every time you run a build pipeline, these artifacts will be generated and stored in the container and artifact registry with a unique string to identify them. In case of a build failure, these artifacts are used to roll back to last successful version.

Important Note: **Do not delete any artifacts and resources created in this lab because they will be required in the upcoming labs.**

Congratulations! You have successfully deployed a Web Application to an OKE cluster using OCI DevOps Build and Deployment pipelines.